MODEL CARD

QR Code Detector v8.2

MODEL DETAILS

Single-class QR bounding-box detector trained on laptop-captured real-world imagery with padded resize to a fixed 256×192 RGB tensor. The LATTE pipeline uses a SqueezeDet architecture with quantization-aware training via LSCQuant, data-driven anchors, and dense detection over a 12×16 grid tuned for prominent near-field QR codes.

MODEL SPECIFICATIONS

Inputs

- Fixed spatial size height × width × channels = 192 × 256 × 3 (`image_size: [192, 256, 3]`)
- RGB input retained in the training/validation augment pipeline (`gray_scale: false`); padding resize (`padding_resize: true`) preserves aspect ratio before letterboxing to the tensor
- Label box format: centered (`cxcywh`, normalised to [0, 1]); use the matching decoder when consuming raw network outputs

Outputs

- SqueezeDet tensors for class logits, localization offsets, and objectness/confidence aligned with anchors (post-process via `interpret_squeezedet_output`)
- Deployment maps decoded boxes back to original image coordinates using preserved padding metadata (`original_size`, `padding`)

Architecture

- Backbone/head: `SqueezeDet` with `use_residual: true`
- Anchor grid: 12 × 16 cells; 4 anchors per cell; anchor side lengths (pixels) [70, 89, 106, 129]
- Single detection class (`num_classes: 1`, class name `qr`)
- Quantization scheme at export: `lsq-default`

Parameters

- 464,888 total parameters (458,712 trainable, 6,176 non-trainable)

AUTHORS	Lattice Semiconductor
VERSION	8.2
RELEASE	Training completed 2026-05-19; HDF5 exported for sensAI deployment
LICENSE	Lattice Semiconductor
ARTIFACT	sensAI HDF5 weights file: qr-code-det-v8.2.h5
TOOLCHAIN	LATTE LSCQuant

TRAINING CONFIGURATION

Training Environment

- Framework: TensorFlow + LATTE + LSCQuant callbacks
- Experiment config: `qr_code_4_3_RGB.yaml` / project `FD_realworld_qr`

Data

- Real-world laptop-webcam QR imagery (single class); labels in centred `cxcywh` format
- Dataset: `dataset/images` — 7,281 train / 910 val / 911 test images
- Each labelled frame contains exactly one QR code; captures are 1920×1080 letterboxed to 256×192

PERFORMANCE EVALUATION

Training metrics (LATTE)

- Primary training objective: weighted composite loss on the validation split.
- `OverlayCallback` during training/visualisation uses `iou_threshold: 0.5` and `prob_threshold: 0.5`.

Offline detection metrics

COCO-style bounding-box mAP via pycocotools (`COCOeval`, area=all, maxDets=100) on the held-out test split (`qr_laptop_images`). Predictions counted toward mAP use score ≥ 0.66; NMS uses IoU 0.25.
- Greedy IoU-matched extras (match threshold 0.50): mean IoU 0.8869, F1 0.8140, image-level FPR 0.0000, TP/FP/FN 70/0/32

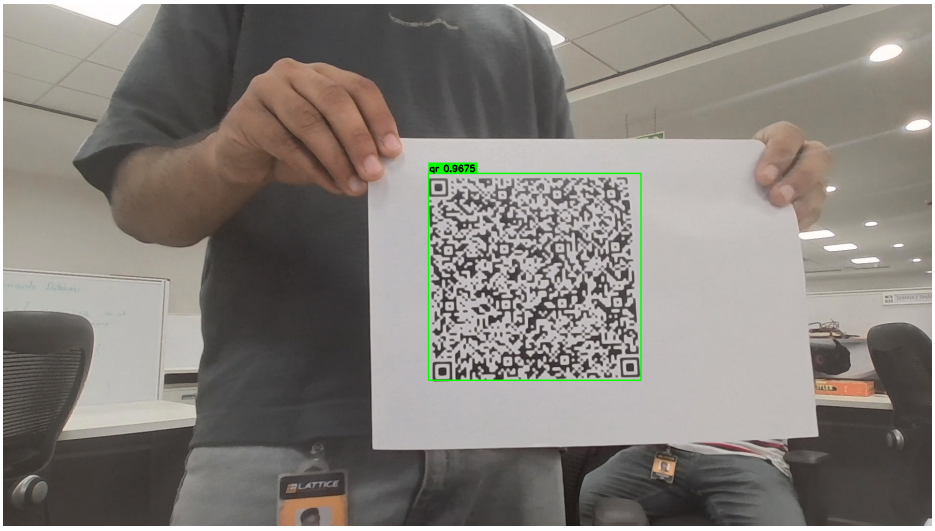
KPIs

Summary metrics for the test-set evaluation (COCO protocol) and training.

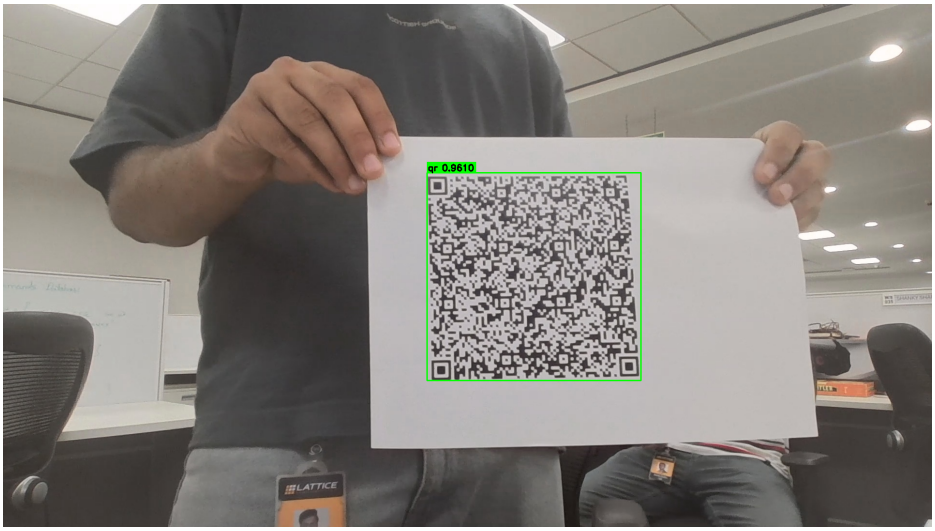
Metric Category	Metric Name	Value	Dataset & Comments
COCO mAP (score ≥ 0.66)	mAP@0.50:0.95	0.5111	qr_laptop_images test split
	mAP@0.50	0.6672	qr_laptop_images test split
	mAP@0.75	0.6071	qr_laptop_images test split

EXAMPLE DETECTIONS

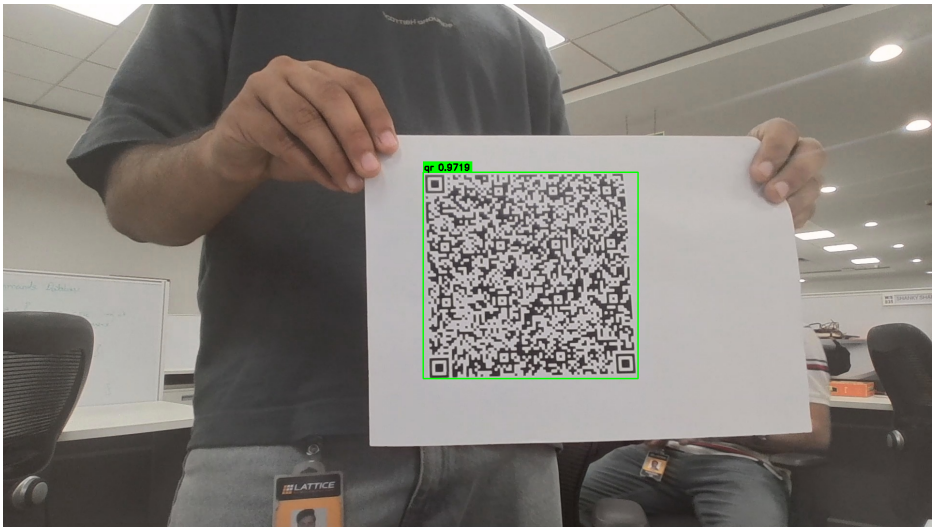
Illustrative test-set frames where the decoder produced a single bounding box after non-maximum suppression (IoU threshold 0.25, score threshold 0.5). Green overlay: predicted box and class score.



Single detection, confidence 0.9675.



Single detection, confidence 0.9610.



Single detection, confidence 0.9719.

CONVERSION & DEPLOYMENT

HDF5 export

- Converter: `sensai-h5` (sensAI HDF5 weights compatible with Lattice sensAI toolchain)
- Checkpoint loaded for export: best training checkpoint at epoch 165 (minimum validation composite loss)

Operating range

- Trained on QR codes occupying roughly 7%–62% of the frame with most images having QR code occupancy of between 15%-20%$ (~51–151 px side length in the 256×192 input)
- Detection of small or distant QR codes below ~7% frame area is out of distribution

Recommendations

- Match LATTE preprocessing and anchor/grid settings exactly when integrating downstream to avoid calibration drift.
- Tune score threshold and NMS on the deployment target when trading off duplicate suppression versus recall.

USAGE

Inference

- Load weights through the LATTE/sensAI toolchain compatible with sensAI HDF5 exports
- Preprocess imagery to 192×256×3 RGB with the same letterboxing semantics as training (scale + replicate padding, divide by 128)
- Decode with `interpret_squeezedet_output` using matched image size, anchor grid [12×16], anchor scales [70, 89, 106, 129], and 1 class